1 CMC Flow and Trajectory Generation

This section implements a time-dependent switching double-gyre flow together with the numerical generation of particle trajectories on a prescribed spatial domain.

The construction follows the growing coherent structure framework described in Section 6.3 of the paper *"An Inflated Dynamic Laplacian to Track the Emergence and Disappearance of Coherent Sets"*.

1.1 Velocity Field Definition

The velocity field is implemented in the file

`generate_pts_CMC.py`,

where the system is defined as a time-dependent ordinary differential equation of the form

$$\frac{d\boldsymbol{x}}{dt} = \boldsymbol{v}(t, \boldsymbol{x}),$$

with $\boldsymbol{x} = (x_1, x_2) \in \mathbb{R}^2$.

The switching double-gyre structure is constructed so that the flow alternates between configurations.

The velocity field is evaluated pointwise during time integration and depends explicitly on time.

1.2 Spatial Discretization and Initial Conditions

The computational domain is defined as a rectangular region

$$I_x \times I_y \subset \mathbb{R}^2.$$

A uniform grid of initial conditions is generated using

$$m_x \quad \text{points in the } x\text{-direction}, \qquad m_y \quad \text{points in the } y\text{-direction}.$$

The total number of particles is therefore

$$N = m_x \cdot m_y.$$

Each grid point serves as an initial condition for a trajectory. This discretization ensures uniform spatial sampling.

1.3 Time Integration

The system is integrated over a finite time interval

$$[0, T],$$

with a fixed timestep Δt. This produces a discrete set of time instances

$$T_{\text{span}} = \{0, \Delta t, 2\Delta t, \ldots, T\}.$$

For each initial condition $\boldsymbol{x}_i$, the trajectory is obtained by solving

$$\frac{d\boldsymbol{X}_i(t)}{dt} = \boldsymbol{v}(t, \boldsymbol{X}_i(t)).$$

A numerical ODE solver is used to compute the trajectories, ensuring stability and accuracy over long integration times.

1.4 Trajectory Data Structure

The computed trajectories are stored in a three-dimensional array

$$\text{pts} \in \mathbb{R}^{2\times N\times T},$$

where:

- the first dimension corresponds to spatial coordinates,
- the second dimension indexes particles,
- the third dimension indexes time steps.

This tensor representation allows efficient access to both spatial and temporal slices of the data.

The trajectories are saved to disk in the file

`Pts_CMC_40x20x76.mat`

2 Inflated Dynamic Laplacian Utilities and Parameter Selection

This section provides the auxiliary routines required for constructing the inflated dynamic Laplacian, as well as the procedures used to select key parameters.

2.1 Overview of Dependencies

The implementation relies on several numerical components:

- diffusion maps for spatial discretization,
- finite-difference schemes for temporal operators,

- sparse linear algebra routines,
- SEBA for sparse eigenbasis approximation,
- visualization tools for interpreting results.

2.2 Temporal Diffusion Parameter

The temporal diffusion parameter a determines the relative strength of diffusion in time compared to space.

Let

$$\tau = T_f - T_0$$

denote the total time span, and let s_1, s_2 denote spatial domain lengths. Define

$$s_{\max} = \max(s_1, s_2).$$

A heuristic lower bound for a is given by

$$a_{\min} = \frac{\tau}{s_{\max}}.$$

This ensures that temporal diffusion is not negligible relative to spatial scales.

The final parameter is chosen as

$$a = \gamma \cdot a_{\min},$$

where γ is a user-defined multiplier.

2.3 Bandwidth Parameter Selection

The kernel bandwidth ε is a critical parameter in diffusion maps.

The kernel is defined as

$$K_{ij}(\varepsilon) = \exp\left(-\frac{\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2}{4\varepsilon}\right).$$

To determine ε, one computes

$$S(\varepsilon) = \frac{1}{N^2} \sum_{i,j} K_{ij}.$$

The optimal value is selected based on the slope

$$\frac{d \log S(\varepsilon)}{d \log \varepsilon}.$$

This method identifies a scale at which the kernel captures meaningful geometric structure.

3 Diffusion Maps and Spacetime Operator Construction

3.1 Diffusion Maps

Diffusion maps provide a discrete approximation of the Laplace–Beltrami operator.

The kernel matrix is given by

$$K_{ij} = \exp\left(-\frac{\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2}{4\varepsilon}\right).$$

This matrix is normalized to produce a Markov operator.

To improve computational efficiency, nearest-neighbor truncation is applied, resulting in a sparse representation.

3.2 Temporal Laplacian

The temporal operator is constructed using second-order finite differences:

$$L_t \approx \frac{\partial^2}{\partial t^2}.$$

This operator captures temporal smoothness in the evolving system.

3.3 Spacetime Diffusion Operator

The temporal diffusion operator is generated from the temporal Laplacian:

$$P_t^{1/2} = \exp\left(\frac{\varepsilon\, t_{\text{factor}}}{2} L_t\right),$$

where L_t is the temporal Laplacian matrix.

The full spacetime operator is constructed as

$$P_a = P_t^{1/2} P_x P_t^{1/2},$$

where:

- P_x is the spatial diffusion operator,
- P_t is the temporal diffusion operator.

3.4 Sparse Eigenbasis Approximation (SEBA)

The file `seba.py` implements Sparse Eigenbasis Approximation (SEBA) to extract sparse coherent structures from eigenvectors.

SEBA (Sparse Eigenbasis Approximation) is used to obtain sparse representations of eigenvectors.

Given an eigenvector matrix

$$V \in \mathbb{R}^{p \times r},$$

a QR decomposition is first applied:

$$V = QR.$$

A soft-thresholding operation is then applied:

$$s_i = \text{sign}(z_i) \max(|z_i| - \mu, 0),$$

where

$$\mu = \frac{0.99}{\sqrt{p}}.$$

After thresholding, a polar decomposition step updates the rotation matrix using singular value decomposition (SVD).

The iteration continues until convergence:

$$\|R_{\text{new}} - R\| < \text{tol}.$$

Finally, the sparse modes are normalized and sorted to produce coherent structure indicators.

This promotes sparsity and enhances interpretability of coherent structures.

4 Inflated Dynamic Laplacian Solver and Visualization

4.1 Eigenvalue Problem

The final step involves solving an eigenvalue problem associated with the spacetime operator.

Eigenvalues are transformed as

$$\Lambda_k = \frac{\log(\nu_k)}{\varepsilon}.$$

4.2 Mode Classification

The solver separates modes into spatial and temporal categories. Modes are classified using temporal averages

$$m_k(t) = \frac{1}{N} \sum_{i=1}^{N} v_k(i,t).$$

The temporal variance

$$\mathrm{Var}(m_k)$$

is used for classification, where low variance indicates spatial modes and high variance indicates temporal modes.

4.3 Visualization

Visualization includes:

- eigenvalue spectra,
- low-dimensional embeddings,
- coherent structure plots,
- trajectory evolution.

4.4 Main Execution Script

The script `run_all.py` orchestrates the workflow:

- loading trajectory data,
- computing parameters,
- constructing operators,
- solving eigenproblems,
- generating visualizations.

All outputs are saved in a structured results directory.